

Test de résumé : Initiation à l'algorithmique

Derrez Lorenzo

September 19, 2026

1^e idée fondamentale : Un algorithme n'est correct que relativement à un problème.

L'idée :

– Un problème est une relation (entrée, sortie), pas une fonction. Un algorithme le résout si, pour toute entrée, toute exécution termine et donne une sortie attendue. Il doit aussi être lisible et de faible complexité, mais une solution rapide et fautive ne vaut rien.

L'argument :

– Plusieurs sorties peuvent être acceptables pour une même entrée. Le non-déterminisme vient de l'aléa ou de la représentation d'une même entrée de plusieurs façons. La correction est donc une propriété de la paire (algorithme, problème), jamais de l'algorithme seul.

Ce qui reste discuté :

– L'exigence de terminaison (correction totale ou partielle) est relâchée en théorie de la complexité. La définition « toute exécution » exclut les algorithmes probabilistes de type Monte Carlo. Une spécification peut aussi être mal posée sans que personne s'en aperçoive.

Piège à éviter :

– Croire qu'un algorithme est correct parce qu'il passe quelques exemples, sans vérifier que la spécification couvre toutes les entrées.

2^e idée fondamentale : Un coût n'a de sens que dans un modèle de calcul explicite.

L'idée :

– La mémoire se divise en CODE, PILE (un bloc par appel, libéré au retour) et TAS (tableaux, libérés explicitement). Un type non primitif a pour valeur une adresse.

L'argument :

– Après $b = a$ sur des tableaux, a et b désignent le même objet. Il se peut alors que la pile se libère et que le tas s'accumule.

Ce qui reste discuté :

– Le choix du modèle influence les conclusions. Le type integer à coût unitaire est irréaliste, alors que Longint a un coût en $\log(n)$. Entre RAM à coût unitaire, coût logarithmique et machine de Turing, l'exposant polynomial peut changer. Les modèles non déterministes et quantiques repoussent encore la question.

Piège à éviter :

- Croire que $b = a$ copie un tableau, ou oublier qu'une « simple découpe » peut copier.

3^e idée fondamentale : Il existe des limites absolues et des limites pratiques.

L'idée :

– Il n'existe pas de méthode universelle pour produire un algorithme correct, ni pour décider si un algorithme termine. Même pour les problèmes décidables, certains résistent à tout algorithme polynomial connu (3SAT, sac à dos).

L'argument :

– Si l'on savait résoudre « problème $\rightarrow$ algorithme correct », on saurait résoudre TERMINAISON. L'argument est une diagonalisation. Conséquence pratique : tests, typage et preuves automatiques sont toujours partiels.

Ce qui reste discuté :

– P versus NP est ouvert, et aucun algorithme polynomial n'est connu pour 3SAT. Le cours attribue l'indécidabilité à Gödel (1931). Historiquement, l'arrêt est plutôt lié à Turing et Church (1936) : Gödel est un cousin (incomplétude). De même, « presque 100 ans » pour 3SAT est à vérifier, car la NP-complétude date de 1971.

Piège à éviter :

- Confondre indécidable (aucun algorithme, même très lent) et difficile (algorithme exponentiel connu).

4^e idée fondamentale : La complexité se mesure en fonction de la taille de l'entrée, asymptotiquement, dans le pire cas.

L'idée :

– La taille d'une entrée est le nombre d'octets pour la représenter. θ est une équivalence à constante près, θ une majoration. La complexité est un maximum sur les entrées de taille au plus n .

L'argument :

– L'exemple « doubler 11055175255055185255 » contre « recopier une phrase 11055175255055185255 fois » sépare linéaire et exponentiel. Le nombre de chiffres compte, pas la valeur. Les constantes dépendent de la machine, donc on ne garde que le terme dominant.

Ce qui reste discuté :

– Pire cas, moyenne ou analyse lissée : QuickSort est $\theta(n^2)$ au pire mais préféré en pratique. La notation asymptotique peut masquer des constantes décisives, et certains algorithmes « galactiques » sont meilleurs en théorie mais inutilisables. Deux fonctions peuvent aussi ne pas être comparables par O .

Piège à éviter :

- Mesurer en fonction de la valeur d'un entier plutôt que de sa taille.

5^e idée fondamentale : La récursivité se comprend par l'arbre des appels : temps $\sim$ nombre de noeuds, espace $\sim$ l'historique de la pile.

L'idée :

- Un chemin racine $\rightarrow$ noeud de l'arbre est l'état de la pile à un instant. L'arbre est donc l'historique de la pile.

L'argument :

– Chaque noeud coûte du temps, mais seuls les noeuds sur un chemin sont en mémoire simultanément. D'où temps (somme sur tous les noeuds) contre espace (max sur un chemin). Pour $f(n) = n^k + a \times f(n/b)$, on compare le travail à la racine (n^k) au nombre de feuilles ($n^{(log a / log b)}$) : trois régimes. Pour $f(n) = g(n) + f(n-1)$, on cherche une primitive de g .

Ce qui reste discuté :

– « Souvent » n'est pas « toujours » : l'espace vaut la hauteur seulement si les blocs sont de taille constante et sans allocation dans le tas. En pratique, la profondeur de pile est limitée, et l'optimisation de la récursion terminale dépend du langage. Récursif ou itératif est aussi un arbitrage de lisibilité.

Piège à éviter :

- Choisir la mauvaise mesure de taille.

6^e idée fondamentale : Décomposer pour recomposer, et savoir choisir le bon problème à résoudre.

L'idée :

– Décomposition, a appels récursifs, recomposition. On impose l'une des deux étapes, l'autre s'en déduit. Souvent il faut résoudre un problème plus général (ajouter des indices, enrichir la sortie) pour être efficace.

L'argument :

– Le tri fusion a une décomposition triviale et une recomposition coûteuse. Le tri par pivot fait l'inverse. Pour le produit matriciel, passer de 8 à 7 appels (Strassen) change l'exposant, alors que le coût de recomposition ne change pas l'ordre de grandeur

Ce qui reste discuté :

– Le tri fusion est optimal en temps dans le modèle par comparaisons, mais pas en espace. Le cours annonce $\theta(\log n)$ pour QuickSort, ce qui suppose de bien gérer la récursion. L'exposant ω du produit matriciel est toujours inconnu

Piège à éviter :

- Croire qu'une décomposition est gratuite alors que subArray copie.

7^e idée fondamentale : La programmation dynamique supprime la redondance en échange de l'espace contre du temps.

L'idée :

- Quand les mêmes sous-problèmes reviennent, on mémorise (entrée $\rightarrow$ sortie) et on ne les recalcule plus.

L'argument :

– L'arbre de f est exponentiel parce que $f(3)$ est recalculé 6 fois. Avec mémoire, les appels « noirs » (nouveaux) sont au plus égaux au nombre d'entrées possibles, soit $\theta(n^2)$ ici. Les « bleus » sont au plus le double. D'où temps $\approx$ nombre d'états $\times$ coût par état, et espace $\approx$ nombre d'états stockés.

Ce qui reste discuté :

– Le « compromis espace-temps » n'est pas une loi : le DP ascendant peut souvent réduire la mémoire, mais on perd alors de quoi reconstruire la solution. Et si le nombre d'états est lui-même exponentiel, la mémoïsation ne sauve rien.

Piège à éviter :

– Mémoïser sans avoir identifié l'espace d'états, ni vérifié que les sous-problèmes se recoupent vraiment.

8^e idée fondamentale : Le glouton construit la solution par choix irrévocables, et il faut savoir quand cela s'arrête de marcher.

L'idée :

– À chaque étape on prend le meilleur choix local. L'écriture est itérative.

L'argument :

– Tri par sélection et tri par tas suivent le même principe glouton (extraire le minimum), mais la structure de données change le coût : $\theta(n^2)$ contre $\theta(n \log n)$. Un algorithme, c'est une idée plus une structure de données. Pour la location de salle, la stratégie correcte se démontre par un argument d'échange.

Ce qui reste discuté :

– On ne sait pas caractériser en général quand le glouton est optimal (les matroïdes couvrent une famille). Le sac à dos résiste, ce qui renvoie à P versus NP. Il est pourtant pseudo-polynomial en la taille de W .

Piège à éviter :

– Adopter un critère glouton « évident » sans preuve ni contre-exemple.

Listes de Questions sur les idées fondamentales :

1. (**Idée 1**) L'algorithme `diviseur_propre` du cours n'est pas correct sur au moins une entrée. Trouve-la. Faut-il corriger l'algorithme ou la spécification ?
2. (**Idée 1**) Un algorithme qui répond correctement avec probabilité 99,9 % (tirage aléatoire) résout-il le problème au sens du cours ? Qu'est-ce qui changerait pour un algorithme qui ne se trompe jamais mais dont le temps d'exécution varie ?
3. (**Idée 2**) Dans le **naïf** de la recherche, à un instant donné, combien de mémoire les sous-tableaux occupent-ils en tout, si tous sont encore vivants sur la pile ? Pourquoi obtient-on $\Theta(n)$ et non $\Theta(n \log n)$?
4. (**Idée 2**) Construis un algorithme qui ne fait que n multiplications avec le type `integer` (coût unitaire) mais dont le résultat a 2^n bits. Que dit-il sur la validité du modèle ?
5. (**Idée 3**) **TERMINAISON** est indécidable, pourtant on prouve chaque jour qu'un tri termine. Quelle est *exactement* la chose impossible ? Précise les quantificateurs.
6. (**Idée 3**) Un collègue annonce un algorithme en $O(n \cdot W)$ pour le sac à dos. Contredit-il le caractère « difficile » du problème ?

7. (**Idée 4**) Le test de primalité par divisions successives jusqu'à $\sqrt{n}$ est-il polynomial ? Réponds en fonction de la taille de l'entrée, en supposant n de type `Longint`.
8. (**Idée 4**) Pourquoi définir la complexité comme le maximum sur les entrées de taille *au plus* n plutôt que de taille *exactement* n ? Quelle propriété obtient-on ?
9. (**Idée 5**) Pour $f(n) = f(n-1) + f(n-2) + 1$, donne les ordres de grandeur du temps et de l'espace. Pourquoi ne coïncident-ils pas ?
10. (**Idée 5**) Résous $f(n) = n + f(n/2) + f(n/3)$ sans la propriété du cours (qui suppose des sous-problèmes de même taille). Quel raisonnement généralise-t-elle ?
11. (**Idée 6**) Peut-on concevoir un tri où la décomposition et la recomposition sont toutes deux en $O(1)$? Sinon, que peut-on dire de leur somme, sans connaître l'algorithme ?
12. (**Idée 6**) Pour le plateau maximal, complète les "à compléter" (`mG`, `mD`). Pourquoi la seule longueur `m` ne suffit-elle pas à recomposer ?
13. (**Idée 7**) Pourquoi mémoriser `triFusion` n'apporte rien alors que cela sauve `f` ? Quel critère vérifies-tu *avant* de coder ?
14. (**Idée 7**) Dans `f3`, `calc` n'utilise que des sous-intervalles de longueur immédiatement inférieure. Peux-tu ramener l'espace à $\Theta(n)$? Que perds-tu ?
15. (**Idée 8**) Location de salle : propose deux critères gloutons plausibles mais faux, avec un contre-exemple minimal. Quel critère fonctionne, et comment le démontrer ?
16. (**Idée 8**) Rendu de monnaie avec les pièces $\{1, 3, 4\}$ pour 6 : que donne le glouton, et quel est l'optimal ? Pourquoi le glouton fonctionne-t-il avec les euros ? Qu'est-ce que cela dit de la frontière entre glouton et programmation dynamique ?
17. (**Bonus**) Ce cours contient au moins cinq erreurs ou imprécisions. Cherche-les dans `triFusionRec`, la complexité annoncée du tri par tas, `f2aux`, la section 1.5 et le tableau comparatif `triMedian/quickSort`.